Proposal

1C. We plan to primarily use Model 2 (Divide and Integrate) with elements of Model 1 (Pair Programming). Since the project has naturally separable components, accelerometer input, LCD rendering, game logic, and RT scheduling, we will divide ownership of each module, agree on interfaces (shared data structures and function signatures) upfront, and integrate early to catch compatibility issues. For trickier parts like collision detection and the RT game loop, we will switch to pair programming to debug together in real time. This hybrid approach lets us work in parallel efficiently while still collaborating closely on the hardest problems.

Q2A. We plan to implement a Chrome Dinosaur Game. The player will tilt the board to make the dinosaur jump over incoming cacti displayed on an LCD, with the game speeding up as the score increases. Our lower bar is a working single-player version with one obstacle type, basic jump mechanics, a score counter, a game over screen, and a buzzer on collision. If possible, we would like to add a multiplayer mode using two boards, along with stretch features such as night mode, ducking for bird obstacles, and LED-based lives.

Q2B. Concurrency will allow obstacle movement, accelerometer input, LCD rendering, buzzer output, and possibly multiplayer communication to run simultaneously. Locks will protect shared game state, such as the dinosaur position, obstacle positions, score, and any communication-related data, from race conditions. Real-time scheduling will be used to control obstacle movement and jump timing precisely, with the game difficulty increasing over time.

Q2C. The board will handle all core computation for the game. This includes reading accelerometer data for jump detection, updating the game state, performing collision detection, adjusting difficulty as the score increases, managing concurrent processes, rendering graphics to the LCD, and triggering the buzzer on game over. If we implement multiplayer, the board will also handle communication between two boards.

Q2D. We plan to test each component individually before integrating the full system. For the accelerometer, we will test that jump gestures are detected reliably without too many false positives. For game logic, we will verify collision detection using known dinosaur and obstacle positions. For real-time scheduling, we will check that obstacle timing changes correctly as difficulty increases. If multiplayer is implemented, we will test that communication between boards is reliable and that both players' game states remain correctly synchronized.

Q2E. Our main concerns are LCD rendering speed under concurrent load, how much accelerometer noise filtering will be needed for reliable jump detection, and whether multiplayer communication between two boards will be feasible within the project timeline. We would also appreciate guidance on recommended LCD libraries or prior-year examples involving the accelerometer, display, or communication between boards.

Q3A: What (if any) external hardware will you require for this project? Do you already have access to this hardware, or do you need to borrow some from the instructor? Please describe fallback options, in case the hardware is not available for use.

Waiting for grade

We will need a small piezo buzzer for the game over sound effect and an LCD display for rendering the game visuals. We do not currently have access to either and would like to borrow both from the instructor. If the buzzer is unavailable, our fallback is to use the onboard LEDs to signal game over instead (flashing red). For the LCD, we would prefer an LCD that can display obstacles like cacti so any LCD that is compatible with our board would be great. We are not sure what works best, any advice would be greatly appreciated.

ECE3140 Final Project Plan

Chrome Dinosaur Game — Physical Jump Detection via IMU

Team: Ariel, Rumman

Architecture Overview

- **FRDM-KL46Z board** strapped to player's body, reads accelerometer data, detects jumps, and sends events over USB serial using concurrency + RT scheduling
- **Laptop** runs a Python pygame script that receives serial events and controls the game visuals
- **Communication protocol:** board sends `"J\n"` on jump detection, `"D\n"` on duck (tilt), `"S\n"` with score-relevant data if needed

---

Division of Work

Ariel owns (Board-side, Part 1):

- Accelerometer setup and raw data reading
- RT scheduling for precise accelerometer sampling
- Concurrency — separate processes for sampling, jump detection, serial output
- Locks protecting shared sensor state

Rumman owns (Board-side, Part 2):

- Jump detection algorithm — threshold tuning, noise filtering
- Serial communication output from board
- Concurrency for serial transmission process
- Integration testing between board and laptop

Both share:

- Physical jump tuning (you both need to actually jump around)
- Integration between board and Python script
- Website, video, README
- Polish and stretch features

Python game (split evenly):

- Ariel: game rendering, sprites, LCD display loop, Cornell theming
- Rumman: game logic, collision detection, score/difficulty scaling, serial reading script

---

Detailed Weekly Plan

Week 1 — Setup and Foundation

Goal: Board sends raw accelerometer data over serial, Python script receives and prints it, basic game window opens

Ariel — Board:

- Set up new MCUXpresso project with all lab files carried over
- Find and read accelerometer examples from prior years / Canvas tutorials (board uses MMA8451Q accelerometer over I2C)
- Write basic accelerometer initialization code — configure I2C, set sensitivity range (suggest ±2g for jump detection)
- Write a simple polling loop that reads X, Y, Z acceleration values
- Print raw values over serial to laptop to verify data is coming through
- Deliverable: serial monitor on laptop shows live accelerometer X/Y/Z values as you move the board

Rumman — Laptop:

- Set up Python environment with `pygame` and `pyserial` libraries
- Write basic serial reading script that connects to the board's COM port and prints incoming data
- Build basic pygame window — static dinosaur sprite on ground, static cactus, scrolling ground line
- Deliverable: game window opens, serial script reads and prints board data

Both together end of week 1:

- Verify serial connection works reliably — board plugged in, Python reads data without dropping bytes
- Agree on final communication protocol (e.g. `"J\n"` for jump, `"D\n"` for duck)
- Agree on accelerometer axis orientation — which axis corresponds to a physical jump

---

Week 2 — Core Implementation

Goal: Fully playable basic game — physical jump makes dino jump, collision detection works, score increases, game over on hit

Ariel — Board:

- Implement RT scheduling for accelerometer sampling — use PIT channel 1 (already set up in Lab 5) to sample accelerometer every 10ms precisely
- Implement concurrency — three concurrent processes:
    - Process 1: samples accelerometer on RT schedule, writes to shared buffer
    - Process 2: reads shared buffer, runs jump detection algorithm, sets jump flag
    - Process 3: monitors jump flag, sends `"J\n"` over serial when jump detected
- Add locks to protect shared accelerometer buffer between Process 1 and Process 2
- Deliverable: board reliably sends `"J\n"` when physically jumped, no false positives at rest

Rumman — Board + Laptop:

- Implement jump detection algorithm on board — detect spike in vertical acceleration axis above threshold (start with ~1.5g), debounce so one jump = one signal
- Tune threshold by strapping board on and jumping repeatedly, adjust until reliable
- On laptop side: connect serial `"J\n"` event to dino jump in pygame
- Implement full game loop — moving cactus, collision detection, score counter, difficulty scaling (cactus speeds up every 5 points), game over screen
- Deliverable: physically jump → dino jumps on screen, game is fully playable end to end

Both together end of week 2:

- Strap board on, play the game together, identify any issues
- Tune jump threshold collaboratively
- Verify concurrency and locks are working — no corrupted sensor data, no missed jumps

---

Week 3 — Polish, Stretch Goals, Documentation

Goal: Polished demo ready, website complete, video recorded

Ariel — Board stretch + documentation:

- Add duck detection — forward tilt on accelerometer sends `"D\n"` to laptop
- Add IMU-based "running speed" — magnitude of horizontal acceleration sent as `"V{value}\n"` to influence game speed (stretch)

- Add buzzer output on game over — board receives `"X\n"` from laptop and triggers buzzer concurrently
- Comment all board code thoroughly
- Write README for board-side code

Rumman — Game stretch + documentation:

- Add duck animation in pygame when `"D\n"` received
- Add bird obstacles that require ducking
- Add night mode (background color flip at score milestone)
- Add Cornell themed sprites — Big Red bear as the dinosaur, Cornell tower as obstacle (stretch)
- Add high score tracker
- Write README for Python-side code

Both together:

- Record 3-5 minute demo video
- Build project website with all required sections
- Final integration test — full playthrough, verify all features work together
- Submit to Gradescope

Key Technical Decisions to Agree On Early

- **Accelerometer axis:** verify which axis (Z most likely) spikes on a physical jump — do this in week 1
- **Jump threshold:** start at 1.5g spike above baseline, tune in week 2
- **Serial baud rate:** suggest 115200 for reliability
- **Communication protocol:**
  - `"J\n"` = jump detected
  - `"D\n"` = duck detected
  - `"X\n"` = game over (laptop → board, triggers buzzer)
- **Sample rate:** 10ms intervals via RT scheduling (100Hz) — fast enough to catch a jump reliably